IUCAA CRAY CX1 supercomputer

Users Guide (v2.1)

1 Introduction

Cray CX1 at the Inter-University Centre for Astronomy & Astrophysics (IUCAA) Pune is a desktop/deskside supercomputer with 1 head node, 1 GPU node and 4 compute nodes. Each node has two 2.67 Ghz Intel Xeon(5650) processors, with every processor having six cores. In total there are 72 cores on the systems. Since there are twelve cores on every node, one can use twelve OpenMP hardware threads. However, the system supports more than 12 OpenMP threads on all the nodes (apart from the head node).

Every node of the system has 24 GB RAM and in total the system has 144 GB RAM.

At present there is around 5 TB disk space available for users on three different partitions, named **/home, /data1/** and **/data2**. Keeping in mind that this is a small system and not more than ten users will be using it, every user can get 100 GB space in the home area. Data of common use (like WMAP and Planck data) is kept in a common area **/data1/cmb_data** and have the right permissions so that users can use it.

The Cray CX1 is equipped with a 16 port RJ-45 10Base T, 100Base T, 1000Base T Gigabit Ethernet Switch, with the data transfer capability of 1 Gbps, supporting Ethernet, Fast Ethernet, and Gigabit Ethernet Data Link Protocol.

The list of nodes is give in the Table (1).

There are many software, packages and libraries already installed in the system and more (open

S. No.	Node	Comments
1	**hpccmb**	Head node or master node
2	**compute-00-00**	Compute node with Nvidia Tesla c2050 Card on board
3	**compute-00-01**	Compute node 1
4	**compute-00-02**	Compute node 2
5	**compute-00-03**	Compute node 3
6	**compute-00-04**	Compute node 4

Table 1: A summary of the nodes on the Cray CX1 system

source) can be installed. At present the system has the following compilers:

1. gcc version 4.1.2 (**gfortran** and **gcc**).
2. Compilers supplied with the *platform* system (**mpicc**, **mpiCC**, **mpif77**, **mpif90** etc.,) are installed in "/opt/platform_mpi/".
3. Open MPI compilers (**mpic++ ,mpicc, mpiCC, mpicxx, mpiexec, mpif77** etc.,) installed in " /data1/software/openmpi".
4. MPICH 2 is installed in "/data1/software/mpich2".
5. Intel compilers (**icc** and **ifort**) are installed in "/opt/intel/". In this area Intel Mathematical Kernel Library (MKL) and Intel Threading Building Blocks (ITBB) are also installed.

Platform LSF (Platform LSF HPC 7) is successfully running on the system and by default all MPI jobs will be assigned in a "batch queue" mode. In general, users are not needed to specify the nodes to which they want to assign the job, however, it may be useful in some cases.

The system already has many scientific packages/libraries pre-installed or have been installed some of them are as follows:

1. **blacs, linpack, scalapack** and **hdf5** supplied with the system are installed in "/opt".
2. **gnuplot, gv, acoread** etc., are installed.
3. A good number of packages including **fftw2, fftw3, pgplot, cfitsio, lapack** are installed in "/data1/software".
4. Some of the packages related to CMBR work (**cmbfast, camb, cosmomc, healpix etc.,**) are already been installed in "/data1/soft_cmbr/" and more will be installed.

2 Job Submission

Job on the system can be submitted using two different modes. In the first case job can be submitted using a *platform* GUI supplied with the system which can be accessed at the following URL from the IUCAA network.

`http://192.168.11.243:8080/platform/`,

it needs a login and password.

Users can directly submit sequential jobs to individual nodes in a normal way. However, MPI jobs must be submitted using LSF job submitting scripts. In general, a job submitting script can have very rich structure, however, the following script is good enough for most of the jobs.

```
cat  cosmomc.lsf
```

```
#BSUB -J cosmomc
#BSUB -o %J.out
#BSUB -e %J.err
#BSUB -n 8
#BSUB -R "span[ptile=4]"
/opt/platform_mpi/bin/mpirun -np 8   ./cosmomc params.ini
```

which can be submitted in the following way:

```
bsub < cosmomc.lsf
```

There are many options which we can provide in our job submission script. Some of the most important are '-J', '-o','-e', and '-n' , which give the name of the job, name of of the out put file, name of of the error file, the number of node respectively. The '-R' option may be used to specify the number of cores per node which we want to use which is a very important consideration in the case when shared memory programming (OpenMP) and distributed memory programming (MPI) are used together like in the case of 'COSMOMC' code. For the complete list of options which you can have in the submission script type 'bsub -help' on the command line.

Once a job is submitted it can be monitored using the usual command **bjobs** and can be killed using the command **bkill**. It is recommended that after the submission user login into the machines on which the jobs are running and check the status using **top** command.

Note that MPI jobs do not print any output on the standard output and one needs to wait the files to be written. However, the standard output can be seen using the command **bpeek** at any stage.

A detailed list of some of the common LSF commands is given in Table (2).

3 Message Passing Interface (MPI)

In the simplest way we can compile our MPI hello world program (hello_world.c,hello_world.f90) in the following way:

```
mpif90 hello_world.f90
```

or for a c program:

```
mpicc hello_world.c
```

and can be executed in the following way:

```
mpirun -np 4 ./a.out
```

1	lsid	Display the LSF release version, the name of the local load sharing cluster and the name of its master LIM host.
2	lsclusters	Display general configuration information about LSF clusters.
3	lsinfo	Display information about the LSF configuration, including available resources, host types and host models.
4	lshosts	Display information about LSF host configuration, including host type, host model, CPU normalization factor, number of CPUs, maximum memory, and available resources.
5	lsplace	Query LIM daemon for a placement decision. This command is normally used in an argument to select hosts for other commands.
6	lsrtasks	Display or update the user's remote or local task lists. These task lists are maintained by LSF on a per user basis to store the resource requirements of tasks. Default: display the task list in multi-column format.
7	lsrun	Submit a task to the LSF system for execution, possibly on a remote host
	lsgrun	Execute a task on the specified group of hosts.
9	lsload	Display load information on hosts, in order of increasing load.
10	lsmon	Full screen LSF monitoring utility displaying dynamic load information of hosts. lsmon supports all the lsload options, plus the additional -i and -L options. It also has run-time options.
11	bsub	Submit a batch job to the LSF system.
12	bkill	Kill a running job.
13	bjobs	See the status of jobs in the LSF queue.
14	bpeek	Access the output and error files of a job.
15	bhist	History of one or more LSF jobs.
16	bqueues	Information about LSF batch queues.
17	bhosts	Display information about the server hosts in the LSF Batch system.
18	bhpart	Display information about host partitions in the LSF Batch system.
19	bparams	Display information about the configurable LSF Batch system parameters.
20	busers	Display information about LSF Batch users and user groups.
21	bugroup	Display LSF Batch user or host group membership.
22	bmodify	Modify the options of a previously submitted job. bmodify uses a subset of the bsub options.
13	bstop	Suspend one or more unfinished batch jobs.
14	bresume	Resume one or more unfinished batch jobs.
15	bchkpnt	Checkpoint one or more unfinished (running or suspended) jobs. The job must have been submitted with bsub -k.
16	brestart	Submit a job to be restarted from the checkpoint files in the specified directory.
17	bmig	Migrate one or more unfinished (running or suspended) jobs to another host. The job must have been submitted with -r or -k options to bsub.
18	btop	Move a pending job to the top (beginning) or bottom (end) of its queue.
19	bswitch	Switch one or more unfinished (running or suspended) jobs from one queue to another
20	bcal	Display information about the calendars in the LSF Job Scheduler system.
21	bdel	Delete one or more unfinished batch jobs.

Table 2: A summary of common LSF commands

which should work fine. However, if get an error message then try to give either the full path of the 'mpicc', 'mpif90' and 'mpirun':

```
/opt/platform_mpi/bin/mpicc hello_world.c
```

and

```
/opt/platform_mpi/bin/mpirun -np 4 ./a.out
```

Note that using 'mpicc' and 'mpif90' to compile MPI program is not the best way (or I will say is the worst way) to use MPI library the reasons are as follows:

- 'mpif90' and 'mpicc' are created using a particular compiler (other options) which you can see using the following command:

  ```
  mpif90 -show
  ```

  which in our case gives the following output:

  ```
  f95 -I/usr/local/include -I/usr/local/include -L/usr/local/lib -lmpichf90
     -lmpichf90 -lmpich -lopa -lmpl -lrt -lpthread
  ```

  This means that when you use command like 'mpif90' you are tied with a particular compile which you or your code may not like at all.

- Using 'mpif90' and 'mpicc' hides the fact that MPI is just a library, just like any other library (and is not a compiler !) and can be used in whatever way we want to use it.

- Not locking MPI library with compiler we have more options which may be quite useful for complex codes which demand a particular version of compiler. For example, COSMOMC (2014) needs 'ifort 13' and if our 'mpif90' has not created using this compiler then there will be problem.

The best (and recommended way) of a compiling a MPI (fortran) program is as follows:

Compile the program as:

```
ifort  HelloWorld.f90  -o HelloWorld -I/opt/platform_mpi/include -L/opt/platform_mpi/lib/linux_amd64/
```

and then execute it with the following script (hello_world.sub):

```
#BSUB -J HelloWorld
#BSUB -o %J.out
#BSUB -e %J.err
#BSUB -n 8
/opt/platform_mpi/bin/mpirun -np 8  ./HelloWorld
```

with

```
bsub < hello_world.sub
```

Note that at present you cannot run the job compiled in the way as discussed above without script. However, if you use 'mpif90' you can run a MPI program with lsf script also:

```
mpif90 HelloWorld.f90  -o HelloWorld
mpirun -np 4 ./HelloWorld
```

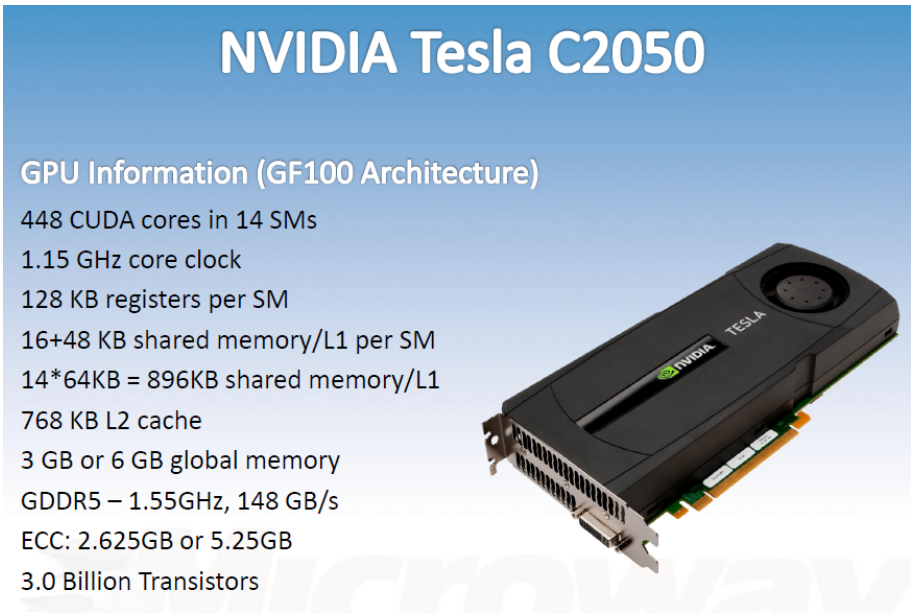

Figure 1: Nvidia Tesla C2050 Graphic Card is hosted on node **compute-00-00**.

should run without any issue but this will not launch the job on other nodes for which you must use the script.

Note that the system has multiple 'mpirun' so check which one you are using if there is any problem.

4 GPU Computing

The system has one Nvidia Tesla C2050 GPU which is hosted on the node **compute-00-00**. Detail status of the GPU can be printed on the standard output using the **nvidia-smi** command one of its example is as follows:

```
nvidia-smi  -q
```

This will print a lot of useful information about the GPU. You can use "–help" option also to find all the option available for **nvidia-smi**.

Compute Unified Device Architecture (CUDA) library as well as the compiler **nvcc** is installed in the are "/usr/local/cuda". In order to run a cuda program, follow the following three steps:

- Write a source/program file with suffix ".cu" following the cuda syntax.
- Compile the program with **nvcc**

  ```
  nvcc prog.cu
  ```

- Run the executable

  ```
  ./a.out
  ```

I have written a program called 'mygpu.cu' if run, gives the following information about the GPU.

```
nvcc mygpu.cc
./a.out
```

which gives

```
 --- General Information for device 0 ---
Name: Tesla C2050
Compute capability: 2.0
Clock rate: 1147000
Device copy overlap: Enabled
Kernel execition timeout : Disabled
 --- Memory Information for device 0 ---
Total global mem: 2817982464
Total constant Mem: 65536
Max mem pitch: 2147483647
Texture Alignment: 512
 --- MP Information for device 0 ---
Multiprocessor count: 14
Shared mem per mp: 49152
Registers per mp: 32768
Threads in warp: 32
Max threads per block: 1024
Max thread dimensions: (1024, 1024, 64)
Max grid dimensions: (65535, 65535, 65535)
```

5 User account on Cray CX1

If you are already working with CMB group at IUCAA and want to have an account then send an email to me [`jayanti@iucaa.ernet.in`] with cc to Prof. Tarun Souradeep [`tarun@iucaa.ernet.in`]. If you have an account on Cray CX1 and want some packages to be installed mail me.

Useful References

- http://www.platform.com/workload-management/high-performance-computing
- http://www-cdf.fnal.gov/offline/runii/fcdfsgi2/lsf_batch.html
- http://www.iucaa.ernet.in/ jayanti/parallel.html